HACKATHON SUBMISSION · SCOREME PLACEMENT DRIVE

Resilient Decision System

A configurable workflow decision platform that validates requests, evaluates versioned rules, executes multi-stage workflows, survives dependency and storage failures, and explains every decision with a tamper-evident audit trail.

85/85	4	17	0
automated tests passing	business workflows · 5 config versions · 0 bespoke code	REST endpoints + full CLI	third-party dependencies (Python stdlib only)

Source code: https://github.com/billbented/resilient-decision-system
Run: `python3 -m unittest discover -s tests -t .` · `python3 -m decision_platform demo` · `python3 -m decision_platform serve`

Candidate: Rajul Tiwari · github.com/billbented

Implementation: Python 3.10+ · SQLite · 2,414 lines of platform code · 873 lines of tests

Contents: 1 Summary · 2 Problem structuring · 3 Architecture · 4 Data flow · 5 Lifecycle · 6 Configuration · 7 Robustness · 8 Audit & explainability · 9 API · 10 Testing · 11 Decision examples · 12 Trade-offs · 13 Assumptions · 14 Scaling · 15 How to run

1 Summary & evaluation map

The platform is one generic engine. Each business process (loan approval, insurance claims, vendor onboarding, employee onboarding) is a versioned JSON workflow. A request is validated, made idempotent, pinned to the workflow version active at intake, and moved stage by stage. A stage optionally calls an external dependency, evaluates its rules and checkpoints atomically. The request ends in `APPROVED`, `REJECTED` or `MANUAL_REVIEW`, or parks in `RETRY_PENDING` when a dependency is down.

Criterion	Marks	How this submission addresses it	See
Problem Structuring	20	Five concerns with one owner each (what/how/when/remember/outside); clear in/out-of-scope; layered packages with a composition root.	§2, §3
Architecture Quality	20	Transport → application → pure domain → infrastructure. Rule engine has no I/O; storage and dependencies swappable in one file.	§3–§5
Engineering Robustness	20	Idempotency (DB-enforced), atomic checkpoints, dependency-result cache, leases, optimistic locking, timeout + jittered retries + circuit breaker, workflow retry schedule, fail-safe fallbacks.	§7
Configurability & Maintainability	15	Workflows are versioned JSON validated at load; hot reload is atomic; version pinning; what-if simulation; one-call rollback; 4 workflows with zero bespoke code.	§6
Explainability & Auditability	10	Every rule records actual vs expected values, guards and data refs; hash-chained audit verified on read; chronological narrative per decision.	§8, §11
Testing Depth	10	85 tests incl. 12-thread duplicate race, mid-transaction storage failure, real timeouts, breaker half-open, audit tampering, HTTP end-to-end.	§10
Trade-off Defence	5	Each major choice listed with the rejected alternative and the reason.	§12

Deliverables checklist

Required deliverable	Where
Working runnable implementation with README	`decision_platform/`, `README.md`
Architecture document (design, components, data flow, trade-offs, assumptions)	`docs/ARCHITECTURE.md` and §2–§5, §12–§14 below
Configuration model	`docs/CONFIGURATION.md`, `configs/*.json`, §6
API interface (REST / CLI)	`decision_platform/api.py`, `__main__.py`, §9
Tests: happy path, invalid input, duplicates, dependency failure, retry, rule change	`tests/` (85 tests), §10
Decision explanation examples	`docs/DECISION_EXAMPLES.md` (14), `examples/explanations/*.json`, §11
Constraints: requirement changes, simulated dependency, idempotency, audit logs, configurable, scaling	§6, §7, §8, §14

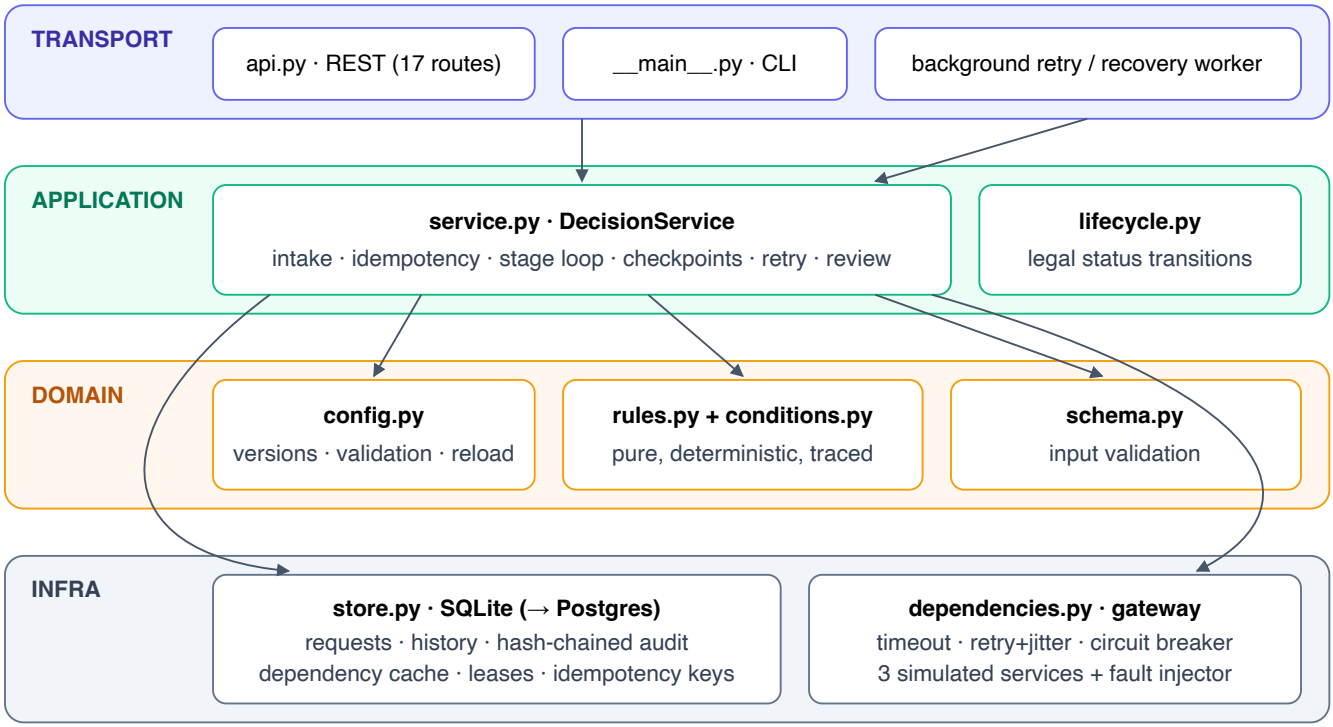
2 Problem structuring

The difficult part is not evaluating an `if`. It is making decisions that stay **correct, repeatable and explainable** while inputs are bad, dependencies fail, clients retry and requirements change. I split the problem into five concerns, each with a single owner, so a change in one never touches the others:

Concern	Question	Owner	Changes when…
What to decide	Which checks, thresholds, branches?	Workflow JSON	business policy changes
How to evaluate	Does this rule pass for this data?	`rules.py`, `conditions.py` (pure)	a new operator is needed
When / order	Next stage? retry? stop?	`service.py`, `lifecycle.py`	orchestration semantics change
Remember	What happened, and can we prove it?	`store.py`	database changes (SQLite → Postgres)
Outside world	How to call flaky services safely?	`dependencies.py`	a real integration replaces a simulator

Out of scope (explicit): authentication and authorisation, multi-tenancy, a rule-authoring UI, and real third-party APIs. They are simulated behind the same interface a real client would implement.

3 Architecture



Dependencies point downward only. The **rule engine is pure** (no I/O, clock or randomness), which is what makes checkpoint/resume safe and decisions reproducible. `app.py` is the composition root, the only module that names concrete classes.

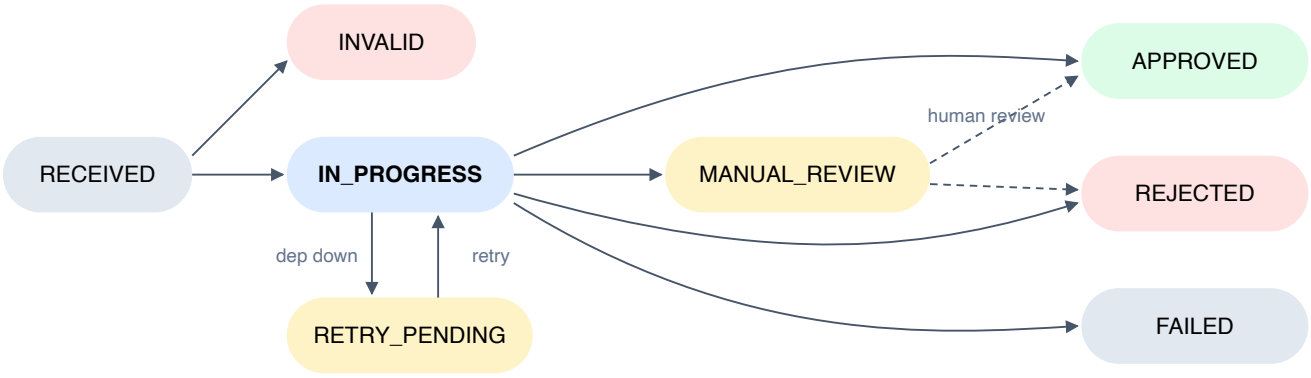
Module	Responsibility
`schema.py`	JSON-Schema subset validator. Returns *all* errors so clients fix a payload in one round trip. Also self-checks schemas at config load.
`conditions.py`	Declarative condition DSL (13 operators, all/any/not, derived thresholds via `{"ref","multiply"}`). Evaluation returns a full trace; a static checker rejects unknown operators or fields.
`rules.py`	Evaluates the rules of a stage (guard → condition → effect). The outcome is the most severe effect: reject > review > pass. Also resolves conditional branches.

`config.py`	Workflow model, load-time validation (references, cycles, reachability, duplicates), immutable versions, atomic hot reload, activation and rollback.
`lifecycle.py`	The state machine. The only code that knows legal status transitions.
`dependencies.py`	Gateway with real timeouts, bounded retries using exponential backoff and full jitter, and a circuit breaker per dependency. Includes 3 simulators and a `FaultInjector` (transient, permanent, timeout, outage, latency).
`store.py`	Transactions, optimistic locking, UNIQUE idempotency, processing leases, hash-chained audit, dependency-result cache, and storage fault injection.
`service.py`	Orchestrator: intake, replay, stage loop, checkpoints, diversion policies, retry and recovery sweep, human review, explain, simulate.
`api.py` / `__main__.py`	Thin transport. Maps typed errors to HTTP codes (404/409/422/503), never leaks stack traces, emits structured logs.

4 Data flow of one request

1. **Intake.** `POST /workflows/loan_application/requests` with an optional `Idempotency-Key`.
2. **Idempotency.** Look up `(workflow, key)`.
   - Same payload hash: return the stored result (HTTP 200, `Idempotent-Replay: true`).
   - Different hash: HTTP 409.
   - No key: the canonical payload hash is the key.
3. **Validate.** Check the schema and collect every error. Invalid requests are persisted as `INVALID`, audited and answered with 422.
4. **Pin and persist.** Store `workflow_version` and `config_hash` with the request. Request row, history and audit are written in one transaction. The UNIQUE constraint settles concurrent duplicates.
5. **Lease.** A conditional UPDATE acquires a processing lease, so only one worker handles a request.
6. **For each stage:**
   - **Fetch.** Reuse a cached dependency result if one exists. Otherwise call the gateway, then persist the result and attempt log in their own transaction.
   - **Evaluate** the rules. The function is pure and traced.
   - **Resolve** the transition.
   - **Checkpoint atomically:** current stage, the `stage.evaluated` audit entry with the full trace, and the decision if terminal.
7. **Dependency down.** The fetch's `on_unavailable` policy decides:
   - `retry`: go to `RETRY_PENDING` with a configured backoff (30 s, 120 s, 600 s) until `on_exhausted`.
   - `review`: fail-safe to a human.
   - `skip`: continue without the data.
8. **Background worker.** Runs due retries and resumes any request whose processor died (expired lease) from its last checkpoint.

5 Request lifecycle



`lifecycle.check()` guards every status change. Terminal states are final: a second review of a decided request returns 409. Each transition writes a `state_history` row with the actor (`api`, `engine`, `retry-worker` or `reviewer:<name>`) and the reason.

6 Configuration model

A workflow is a JSON document containing:

- an input schema;
- declared dependencies, each with a timeout and retry policy;
- a workflow-level retry policy;
- stages, each with fetches, rules (optional `when` guard, `condition`, `on_fail` of reject, review or flag) and transitions (fixed, or ordered conditional branches with a default).

Below is the `affordability` stage of `loan_application`. It has a derived threshold, a guarded rule and conditional branching:

```
{
 "description": "Multi-step affordability checks with derived thresholds",
 "rules": [
  {
   "id": "R-AFF-001",
   "type": "composite",
   "description": "Loan must not exceed 5x annual income",
   "condition": {
    "field": "input.loan_amount",
    "op": "lte",
    "value": {
     "ref": "input.annual_income",
     "multiply": 5
    }
   },
   "on_fail": "reject"
  },
  {
   "id": "R-AFF-002",
   "type": "conditional",
   "description": "Existing debt should be <= 50% of income",
   "when": {
    "field": "input.existing_debt",
    "op": "exists"
   },
   "condition": {
    "field": "input.existing_debt",
    "op": "lte",
```

```
    "value": {
     "ref": "input.annual_income",
     "multiply": 0.5
    }
   },
   "on_fail": "review"
  }
 ],
 "transitions": {
  "pass": [
   {
    "when": {
     "field": "input.loan_amount",
     "op": "gt",
     "value": 2000000
    },
    "to": "senior_approval"
   },
   {
    "to": "APPROVED"
   }
  ]
 }
}
```

A requirement change, end to end (no code touched)

Risk asks to tighten straight-through approval. `loan_application.v2.json` is added:

Rule	v1	v2
`R-CRD-003`	`deps.credit.score gte 650`	`deps.credit.score gte 700`
`R-AFF-002`	high existing debt → `flag`	high existing debt → `review`

1. **Preview.** `POST /workflows/loan_application/simulate?version=1|2` shows the same applicant moving from APPROVED to MANUAL_REVIEW. Nothing is persisted.
2. **Deploy.** `POST /admin/reload`. Reload is atomic: an invalid file is rejected and the old config stays live.
3. **In-flight requests** stay pinned to v1, which is tested even across a retry.
4. **Rollback.** `POST /admin/workflows/loan_application/activate {"version":1}`.

Static validation at load catches all of the following:

- unknown operators and bad regexes;
- fields outside the `input.`, `deps.` and `meta.` namespaces;
- dangling stage targets, cycles and unreachable stages;
- branch lists without a default;
- duplicate rule ids and undeclared dependencies;
- edits to an already-published version.

Adding a new business process is a new file. `employee_onboarding` required zero lines of code.

7 Engineering robustness

Failure	Mechanism	Outcome (tested)
Invalid / malformed input	Schema validation (all errors), JSON parse guard, 1 MB body cap	422; persisted as INVALID; audited
Duplicate request	Idempotency key + canonical payload hash	Original result, no re-processing

Concurrent duplicates	DB UNIQUE(workflow, key) + processing lease	12 threads → 1 record, 1 bureau call
Key reused, new payload	Hash comparison	409 idempotency_conflict
Transient dependency error / timeout	Real deadline; retries with exponential backoff + full jitter (capped)	Absorbed within the call
Sustained outage	Circuit breaker (closed → open → half-open); workflow `RETRY_PENDING` with backoff schedule	Fails fast; auto-recovers
Retries exhausted	`retry_policy.on_exhausted`	MANUAL_REVIEW (never stuck)
Permanent dependency error	Not retried; doesn't trip breaker	Straight to review
Partial save (failure mid-transaction)	One transaction per checkpoint; rollback	Stays at last good stage; resumes; each stage audited exactly once
Crash after an external call	Per-request dependency-result cache, written immediately	Resume never re-calls the service
Worker dies mid-flight	Lease with TTL + recovery sweep	Another worker resumes
Lost update	Optimistic locking (`row_version`)	ConcurrencyConflict, not an overwrite
Bad config / infinite loop	Load-time validation, cycle detection, `max_steps`	Rejected before it can decide anything

> **Why resume is safe:** a decision is a pure function of (config version, input, dependency results), and all three are persisted. Re-running a stage after any failure therefore yields the same answer, and the atomic checkpoint ensures it is recorded once.

8 Auditability & explainability

- **Hash-chained audit.** Entries are chained per request: `hash = sha256(prev_hash | seq | event | stage | data | ts)`. Every read of `/audit` re-verifies the chain. A test edits one row and gets `broken_at_seq: 3`.
- **Rule traces.** `stage.evaluated` stores, per rule: actual value, expected value, derived-threshold source, guard result, effect and `data_refs` (exactly which fields the rule read).
- **Dependency records.** `dependency.called` and `dependency.failed` store input, output and every attempt (outcome, error, backoff ms).
- **Reproducibility.** Every request carries `workflow_version` and `config_hash`, so a decision can be replayed against the exact rules that made it.
- **Human decisions.** Manual decisions record reviewer, notes and the system's original recommendation.

Audit events for a request that hit a bureau outage and recovered, in order:

```
request.received → validation.passed → stage.evaluated[eligibility] → dependency.failed[credit_assessment]
→ retry.scheduled → retry.started → dependency.called → stage.evaluated[credit_assessment]
→ stage.evaluated[affordability] → decision.made          (hash chain VALID, eligibility not re-run)
```

9 API & CLI

Method	Path	Purpose
POST	`/workflows/{wf}/requests`	Submit. 201 created · 200 replay · 409 conflict · 422 invalid
POST	`/workflows/{wf}/simulate?version=N`	What-if / rule-change impact (no persistence)
GET	`/workflows`, `/workflows/{wf}`	Workflows, versions, definitions
GET	`/requests/{id}` · `/history` · `/audit` · `/explain`	State · lifecycle · audit + integrity · narrative
POST	`/requests/{id}/retry` · `/review`	Manual retry · human approve/reject
POST	`/admin/reload` · `/admin/workflows/{wf}/activate`	Hot reload · pin/rollback version
POST	`/admin/retry-due` · `/admin/faults/{dep}`	Recovery sweep · chaos (outage, fail_next, latency)
GET	`/admin/dependencies` · `/health`	Breaker state, call counts · liveness

The CLI mirrors the API: `submit`, `show`, `explain`, `audit`, `history`, `review`, `retry`, `retry-due`, `simulate`, `validate-config`, `workflows`, `demo` and `serve`.

10 Testing

Result: `Ran 85 tests in 0.733s` , **OK**, with 85 of 85 passing. Tests run against a fake clock and fault injectors, so failure scenarios are deterministic and the whole suite takes about one second. The API tests use a real HTTP server.

Required scenario	Representative tests
Happy path	auto-approval across 3 stages; claim fast-track; vendor and onboarding approvals; HTTP submit → explain → audit
Invalid input	all errors reported at once; bool ≠ integer; non-object body; malformed JSON → 422; nothing downstream runs
Duplicate requests	replay returns same id; 409 on payload change; key-order independence; payload-hash fallback; 12-thread race → 1 record & 1 dependency call
Dependency failure	transient ×2 then success (backoff verified); permanent error → review; real timeout; circuit breaker opens, fails fast, half-open recovers
Retry flow	outage → RETRY_PENDING (30 s schedule) → sweep resumes at checkpoint; exhausted after 3 → MANUAL_REVIEW
Rule change	v1 approves / v2 reviews the same payload; in-flight pinned to v1 across retry; simulate; rollback; new workflow from config only
Consistency	mid-transaction storage failure rolls back and resumes exactly once; intake failure leaves no residue; optimistic lock; lease

Workflow execution (17)

- ✓ `cannot_review_non_review_request`
- ✓ `conditional_branching_to_senior_stage`
- ✓ `conditional_rule_applies_only_when_guard_true`
- ✓ `cross_field_reference`
- ✓ `derived_threshold_reject`
- ✓ `flags_do_not_block`
- ✓ `happy_path_approved`
- ✓ `invalid_input`
- ✓ `manual_review_then_human_decision`
- ✓ `non_object_payload`
- ✓ `reject_on_mandatory_rule`
- ✓ `reject_on_threshold`
- ✓ `review_requires_reviewer`
- ✓ `second_workflow_same_engine`
- ✓ `senior_stage_composite_rule`
- ✓ `unknown_workflow_and_request`
- ✓ `vendor_and_onboarding`

Idempotency & duplicates (7)

- ✓ `concurrent_duplicates_create_exactly_one_request`
- ✓ `invalid_request_replay_is_stable`
- ✓ `key_order_does_not_matter`
- ✓ `keys_are_scoped_per_workflow`
- ✓ `no_key_falls_back_to_payload_hash`
- ✓ `same_key_different_payload_conflicts`
- ✓ `same_key_same_payload_replays`

Failures, retries, breaker, storage (15)

- ✓ `half_open_failure_reopens`
- ✓ `opens_fails_fast_and_half_opens`
- ✓ `backoff_schedule_from_config`
- ✓ `dependency_result_reused_not_recalled`
- ✓ `fail_safe_policy_review`
- ✓ `manual_retry_only_from_retry_pending`
- ✓ `outage_parks_request_then_retry_succeeds`
- ✓ `permanent_error_goes_straight_to_review`
- ✓ `real_timeout_enforced`
- ✓ `retries_exhausted_goes_to_configured_fallback`
- ✓ `transient_failures_absorbed_by_in_call_retries`
- ✓ `intake_failure_leaves_nothing_behind`
- ✓ `optimistic_lock_detects_lost_update`
- ✓ `partial_save_rolls_back_and_resumes`
- ✓ `processing_lease_blocks_second_worker`

Rule change & audit (10)

- ✓ `audit_records_full_trace_and_data_refs`
- ✓ `explanation_contents`
- ✓ `history_is_complete`
- ✓ `tampering_breaks_hash_chain`
- ✓ `in_flight_request_stays_pinned_to_its_version`
- ✓ `new_workflow_is_pure_config`
- ✓ `rollback_by_activation`
- ✓ `simulate_previews_impact_and_persists_nothing`
- ✓ `simulate_with_supplied_dependency_outputs`
- ✓ `threshold_change_changes_decision_without_code`

REST API (end-to-end) (6)

- ✓ `duplicate_returns_200_with_replay_header`
- ✓ `invalid_input_422`
- ✓ `malformed_json_and_unknown_routes`
- ✓ `review_flow_and_faults_admin`
- ✓ `submit_explain_audit`
- ✓ `workflows_and_simulate`

Configuration & registry (14)

- ✓ `bad_operator`
- ✓ `branches_need_default`
- ✓ `cycle_rejected`
- ✓ `definition_raises`
- ✓ `duplicate_rule_id`
- ✓ `shipped_configs_are_valid`
- ✓ `undeclared_dependency`
- ✓ `unknown_transition_target`
- ✓ `unreachable_stage`
- ✓ `activate_rollback`
- ✓ `bad_reload_is_atomic`
- ✓ `editing_a_published_version_in_place_is_refused`
- ✓ `hot_reload_adds_version`
- ✓ `unknown_workflow`

Rules, conditions & schema (16)

- ✓ `composites_trace_every_branch`
- ✓ `derived_threshold_reference`
- ✓ `missing_field_is_false_not_error`
- ✓ `operators`
- ✓ `static_check_rejects_bad_conditions`
- ✓ `type_mismatch_is_false_with_note`
- ✓ `bool_is_not_integer`
- ✓ `reports_all_errors_at_once`
- ✓ `schema_self_check`
- ✓ `valid`
- ✓ `conditional_branch`
- ✓ `deterministic`
- ✓ `guard_applies_rule`
- ✓ `guard_skips_rule`
- ✓ `reject_outranks_review`
- ✓ `review_default_target`

11 Decision explanation examples

All 14 scenarios are produced by `python3 -m decision_platform demo`, which uses a fixed clock and fixture-backed dependencies. The full input, trace and audit for each is in `docs/DECISION_EXAMPLES.md` and `examples/explanations/*.json`.

#	Scenario	Workflow	Output	Decisive rules
1	Happy path: loan auto-approved	`loan_application` v2	APPROVED	—
2	Threshold miss → manual review → human approval	`loan_application` v2	APPROVED	R-CRD-003
3	Hard reject: poor credit	`loan_application` v2	REJECTED	R-CRD-001, R-CRD-002, R-CRD-003
4	Invalid input rejected at intake (422)	`loan_application` v2	INVALID	—
5	Duplicate request replayed (idempotency)	`loan_application` v2	APPROVED	—
6	Flaky dependency absorbed by retries	`loan_application` v2	MANUAL_REVIEW	R-CRD-002
7	Outage → RETRY_PENDING → auto-retry succeeds	`loan_application` v2	APPROVED	—
8	Rule change v1→v2 (no code change)	`loan_application` v2	MANUAL_REVIEW	R-CRD-003
9	Large loan branched to senior approval	`loan_application` v2	APPROVED	—
10	Insurance claim fast-tracked	`insurance_claim` v1	APPROVED	—
11	Claim rejected: tampered document	`insurance_claim` v1	REJECTED	R-DOC-001, R-DOC-002
12	Vendor rejected: sanctions hit	`vendor_approval` v1	REJECTED	R-VEN-001
13	Screening down → fail-safe review	`vendor_approval` v1	MANUAL_REVIEW	—
14	Onboarding: conditional NDA rule	`employee_onboarding` v1	MANUAL_REVIEW	R-EMP-003

Example A: threshold miss → manual review → human approval

Input (applicant `APP-1002`, bureau score 680) → **Output** APPROVED after review.

```
Request 11bec0cd-cd3c-47be-8979-6ca9ca7c0501 | workflow loan_application v2 (config 8e695389fb8d) | status
APPROVED
Stage 'eligibility': 3 passed, 0 failed, 1 skipped -> outcome PASS -> credit_assessment
   [PASS] R-ELG-001 KYC must be verified: input.kyc_verified=True == True -> true
   [PASS] R-ELG-002 Applicant age must be 21-65: input.age=34 between [21, 65] -> true
   [PASS] R-ELG-003 Applicant must have an income source: input.employment_status='salaried' not in
['unemployed'] -> true
   [SKIP] R-ELG-004 Self-employed applicants need income >= 5,00,000: not applicable:
input.employment_status='salaried' == 'self_employed' -> false
Dependency credit_bureau consulted at stage 'credit_assessment' (1 attempt): {'bureau_ref': 'CB-682DC6BF',
'open_defaults': 0, 'score': 680}
Stage 'credit_assessment': 2 passed, 1 failed, 0 skipped -> outcome REVIEW -> MANUAL_REVIEW
   [PASS] R-CRD-001 Bureau score must be >= 550: deps.credit.score=680 >= 550 -> true
   [PASS] R-CRD-002 No open defaults on record: deps.credit.open_defaults=0 == 0 -> true
   [FAIL] R-CRD-003 Score >= 700 for straight-through approval: deps.credit.score=680 >= 700 -> false =>
REVIEW
Engine outcome MANUAL_REVIEW driven by: R-CRD-003 (review)
Manual review by credit.officer@bank: APPROVED (system recommended MANUAL_REVIEW). Notes: Salary slips
verified offline; score near threshold.
Audit trail: 7 entries, hash chain VALID
```

Example B: dependency outage → RETRY_PENDING → automatic recovery

```
Request 1080c2e2-cc47-4ff2-82ff-22ba17c5a441 | workflow loan_application v2 (config 8e695389fb8d) | status
APPROVED
Stage 'eligibility': 3 passed, 0 failed, 1 skipped -> outcome PASS -> credit_assessment
   [PASS] R-ELG-001 KYC must be verified: input.kyc_verified=True == True -> true
   [PASS] R-ELG-002 Applicant age must be 21-65: input.age=34 between [21, 65] -> true
   [PASS] R-ELG-003 Applicant must have an income source: input.employment_status='salaried' not in
['unemployed'] -> true
   [SKIP] R-ELG-004 Self-employed applicants need income >= 5,00,000: not applicable:
input.employment_status='salaried' == 'self_employed' -> false
Dependency credit_bureau FAILED at stage 'credit_assessment' after 3 attempt(s): credit_bureau: retries
exhausted after 3 attempts -> policy 'retry'
Workflow retry #1 of 3 scheduled in 30s
Workflow retry #1 started by retry-worker (resumes at the checkpointed stage)
Dependency credit_bureau consulted at stage 'credit_assessment' (1 attempt): {'bureau_ref': 'CB-6FA93833',
'open_defaults': 0, 'score': 742}
Stage 'credit_assessment': 3 passed, 0 failed, 0 skipped -> outcome PASS -> affordability
   [PASS] R-CRD-001 Bureau score must be >= 550: deps.credit.score=742 >= 550 -> true
   [PASS] R-CRD-002 No open defaults on record: deps.credit.open_defaults=0 == 0 -> true
   [PASS] R-CRD-003 Score >= 700 for straight-through approval: deps.credit.score=742 >= 700 -> true
Stage 'affordability': 2 passed, 0 failed, 0 skipped -> outcome PASS -> APPROVED
   [PASS] R-AFF-001 Loan must not exceed 5x annual income: input.loan_amount=1400000 <= 6000000
(input.annual_income x5) -> true
   [PASS] R-AFF-002 Existing debt should be <= 50% of income: input.existing_debt=150000 <= 600000.0
(input.annual_income x0.5) -> true
Decision APPROVED: every applicable rule passed
Audit trail: 10 entries, hash chain VALID
```

Example C: conditional branching to senior approval

```
Request 8a947757-1d99-42ea-bcfe-b18e5a29dfd4 | workflow loan_application v2 (config 8e695389fb8d) | status
APPROVED
Stage 'eligibility': 3 passed, 0 failed, 1 skipped -> outcome PASS -> credit_assessment
Dependency credit_bureau consulted at stage 'credit_assessment' (1 attempt): {'bureau_ref': 'CB-04C35D19',
'open_defaults': 0, 'score': 801}
Stage 'credit_assessment': 3 passed, 0 failed, 0 skipped -> outcome PASS -> affordability
   [PASS] R-CRD-001 Bureau score must be >= 550: deps.credit.score=801 >= 550 -> true
   [PASS] R-CRD-002 No open defaults on record: deps.credit.open_defaults=0 == 0 -> true
   [PASS] R-CRD-003 Score >= 700 for straight-through approval: deps.credit.score=801 >= 700 -> true
Stage 'affordability': 2 passed, 0 failed, 0 skipped -> outcome PASS -> senior_approval via branch
[input.loan_amount=4500000 > 2000000 -> true]
   [PASS] R-AFF-001 Loan must not exceed 5x annual income: input.loan_amount=4500000 <= 30000000
(input.annual_income x5) -> true
   [PASS] R-AFF-002 Existing debt should be <= 50% of income: input.existing_debt=0 <= 3000000.0
(input.annual_income x0.5) -> true
Stage 'senior_approval': 1 passed, 0 failed, 0 skipped -> outcome PASS -> APPROVED
   [PASS] R-SNR-001 Large loans need score >= 750 and salaried employment: (deps.credit.score=801 >= 750 ->
true AND input.employment_status='salaried' == 'salaried' -> true)
Decision APPROVED: every applicable rule passed
Audit trail: 8 entries, hash chain VALID
```

Example D: invalid input

Input

```
{
 "applicant_id": "12345",
 "full_name": "X",
 "age": "thirty",
 "loan_amount": -5,
 "employment_status": "pilot"
}
```

Explanation

```
Request 40cb0bbc-18dd-48a2-b20b-64679588f7e6 |
workflow loan_application v2 (config 8e695389fb8d) |
status INVALID
Rejected at intake: input failed schema validation:
  - $.annual_income: is required
  - $.kyc_verified: is required
  - $.applicant_id: does not match pattern ^APP-[0-
9]{4,}$
  - $.full_name: length must be >= 2
  - $.age: expected integer, got string
  - $.loan_amount: must be > 0, got -5
  - $.employment_status: must be one of ['salaried',
'self_employed', 'unemployed', 'student'], got
'pilot'
Audit trail: 2 entries, hash chain VALID
```

Example E: vendor screening unavailable → fail-safe review

```
Request 8e551818-7cd3-44b4-81f2-7755fcf2fe2c | workflow vendor_approval v1 (config 8e124e6f99c1) | status
MANUAL_REVIEW
Dependency sanctions_screening FAILED at stage 'compliance' after 3 attempt(s): sanctions_screening: retries
exhausted after 3 attempts -> policy 'review'
Decision MANUAL_REVIEW: sanctions_screening: dependency unavailable (policy=review); routed to a human
Audit trail: 4 entries, hash chain VALID
```

12 Trade-offs

Decision	Alternative	Why
JSON condition DSL	Embedded expressions / scripting; rules-engine library	No code execution, validated at load, diffable between versions, explainable value by value. Less expressive, but derived thresholds and composites cover the use cases.
Acyclic stage graph	Allow loops	Guaranteed termination. Repetition (retries) belongs to the lifecycle, not the graph.
Stage-level checkpoints	Per-rule, or none	Rules are pure and cheap, so re-running a stage is free. Fewer writes than per-rule.
Persist dependency results immediately	Persist at checkpoint	Never double-call a paid or side-effecting service after a crash.
Synchronous submit + async retry worker	Fully async queue	One round trip for the common case, async only for slow paths. The queue is the scale-out step (§14).
SQLite + stdlib	Postgres + FastAPI + Redis	Zero setup for reviewers. Transactions, UNIQUE and conditional UPDATE map one-to-one onto Postgres.
Payload hash as fallback key	Mandatory key	Protects naive clients from double submits. Cost: two identical payloads merge, which is accepted because clients can send keys.
Evaluate every rule (no short-circuit)	Stop at first failure	Explanations show every reason. The cost is negligible.
Fail-safe fallbacks (→ MANUAL_REVIEW)	Auto-reject / auto-approve	When information is missing a human decides. Configurable per workflow.
Immutable config versions	Edit in place	Reproducible decisions and safe pinning of in-flight requests. Rollback is a pointer change.

13 Assumptions

- Requests are JSON objects of at most 1 MB. Seconds of latency are acceptable on the retry path.
- External dependencies are read-only lookups, so caching their result per request is valid.
- A human reviewer is available for `MANUAL_REVIEW`. Reviewer identity comes from the caller, since auth is out of scope.
- Clock skew between workers is below the lease TTL (60 s).
- Config authors are trusted, but configs are still validated, because typos are the common failure.

14 Scaling considerations

Pressure	Approach
Throughput	API and workers are stateless; run N replicas. Leases and the UNIQUE constraint already make concurrent workers safe. On Postgres, use `SELECT … FOR UPDATE SKIP LOCKED` for the sweep.
Slow dependencies	Return 202 + `request_id` and process from a queue (Kafka/SQS) partitioned by request id. The resumable stage loop maps directly onto a consumer.
Dependency protection	Breakers already exist per dependency. Add bulkheads (separate pools), per-dependency rate limits, and shared breaker state via Redis.

Audit growth	The log is append-only: partition by month and archive to object storage. Hash chains are per request, so partitions are independent.
Read load	Explanations derive from immutable rows: use read replicas and caching.
Config distribution	Immutable versions published to a config store and cached by (workflow, version). Reload is atomic.
Observability	Structured request logs exist. Add metrics per workflow, stage and rule (decision mix, review rate, dependency latency, breaker state, retry backlog) and alerts.

15 How to run

```
git clone https://github.com/billbented/resilient-decision-system
cd resilient-decision-system
python3 -m unittest discover -s tests -t . -v      # 85 tests, ~1 s
python3 -m decision_platform demo                  # 14 scenarios -> docs/DECISION_EXAMPLES.md
python3 -m decision_platform serve --port 8080     # REST API + retry worker

curl -X POST localhost:8080/workflows/loan_application/requests \
     -H 'Idempotency-Key: k1' -d @examples/payloads/loan_review.json
curl localhost:8080/requests/<id>/explain
```

Requirements: Python 3.10+. No packages to install.